



TaskFlow — Docker Projects Guide

Manual Docker & Docker Compose — Full-Stack Application



Folder Structure

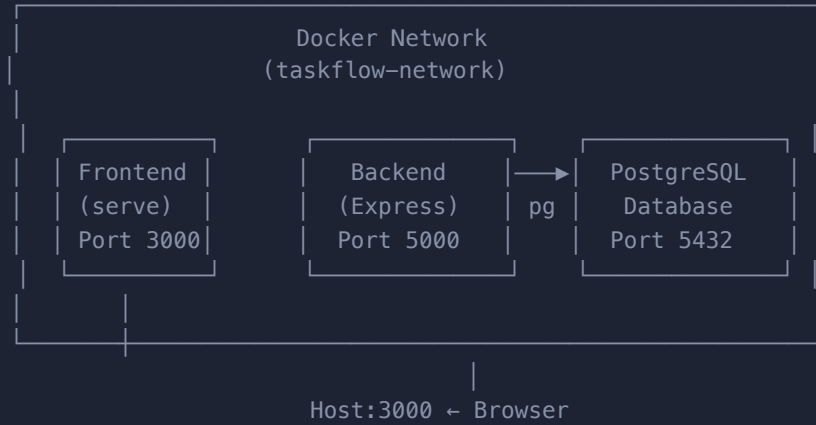
Project 1 — Manual Docker (No Compose)

```
project-1-manual-docker/
├── backend/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── .env.example
│   ├── package.json
│   ├── scripts/
│   │   └── wait-for-it.sh    # Waits for DB before starting
│   └── src/
│       ├── index.js         # Express entry point
│       └── db/
│           ├── connection.js # PostgreSQL pool
│           ├── migrate.js    # Create tables
│           └── seed.js       # Sample data
│       └── routes/
│           ├── health.js     # GET /api/health
│           └── tasks.js      # Full CRUD /api/tasks
└── frontend/
    ├── Dockerfile
    ├── .dockerignore
    ├── index.html
    ├── package.json
    ├── vite.config.js
    └── src/
        ├── main.jsx, App.jsx, api.js
        └── components/ (TaskBoard, TaskModal, StatsBar)
```

Project 2 — Docker Compose

```
project-2-docker-compose/
├── docker-compose.yml    # ★ One-command orchestration
├── .env.example
├── database/
│   └── init.sql          # Auto-runs on first start
├── backend/              # Same code as Project 1
│   ├── Dockerfile
│   ├── package.json
│   └── scripts/ & src/
└── frontend/             # Same code as Project 1
    ├── Dockerfile
    ├── package.json
    └── src/
```

How Services Connect



Request Flow

- Browser → `http://localhost:3000`
 - serve delivers React SPA (static files)
- React app calls `fetch('http://localhost:5000/api/tasks')`
 - Express processes request
 - pg pool queries PostgreSQL
 - Response flows back to browser

API Endpoints

Method	Endpoint	Description
GET	<code>/api/health</code>	Health check + DB status
GET	<code>/api/tasks</code>	List all tasks (filter/sort)
GET	<code>/api/tasks/:id</code>	Get single task
POST	<code>/api/tasks</code>	Create new task
PUT	<code>/api/tasks/:id</code>	Full update
PATCH	<code>/api/tasks/:id</code>	Partial update
DELETE	<code>/api/tasks/:id</code>	Delete task

Project 1 — Manual Docker Setup

Step 1: Create a Docker Network

```
docker network create taskflow-network
```

 **Note:** Docker networks allow containers to discover each other **by container name**. Without a shared network, containers are isolated and cannot communicate.

Step 2: Start PostgreSQL Database

```
docker run -d \  
  --name taskflow-db \  
  --network taskflow-network \  
  -e POSTGRES_DB=taskflow \  
  -e POSTGRES_USER=taskflow_user \  
  -e POSTGRES_PASSWORD=taskflow_secret_2024 \  
  -p 5432:5432 \  
  -v taskflow-pgdata:/var/lib/postgresql/data \  
  postgres:16-alpine
```

Flags Explained

Flag	Purpose
<code>-d</code>	Run in background (detached)
<code>--name taskflow-db</code>	Container name (used as hostname on the network)
<code>--network taskflow-network</code>	Attach to our custom network
<code>-e POSTGRES_*</code>	Environment variables for DB initialization
<code>-p 5432:5432</code>	Map host port to container port
<code>-v taskflow-pgdata:...</code>	Persist data in a named volume

Step 3: Verify Database is Running

```
# Check container status  
docker ps --filter name=taskflow-db  
  
# Check logs  
docker logs taskflow-db  
  
# Test connection from host  
docker exec taskflow-db pg_isready -U taskflow_user -d taskflow
```

Step 4: Build and Run the Backend

```
# Navigate to backend directory
cd project-1-manual-docker/backend

# Build the image
docker build -t taskflow-backend:1.0 .

# Run the container
docker run -d \
  --name taskflow-backend \
  --network taskflow-network \
  -e NODE_ENV=production \
  -e APP_PORT=5000 \
  -e DB_HOST=taskflow-db \
  -e DB_PORT=5432 \
  -e DB_NAME=taskflow \
  -e DB_USER=taskflow_user \
  -e DB_PASSWORD=taskflow_secret_2024 \
  -e CORS_ORIGIN=http://localhost:3000 \
  -p 5000:5000 \
  taskflow-backend:1.0
```

⚠ **Important:** `DB_HOST=taskflow-db` is the **container name** of the database, which Docker DNS resolves within the shared network. This is NOT `localhost` !

Step 5: Verify Backend

```
# Check logs
docker logs -f taskflow-backend

# Test health endpoint
curl http://localhost:5000/api/health

# Test tasks endpoint
curl http://localhost:5000/api/tasks
```

Step 6: Build and Run the Frontend

```
# Navigate to frontend directory
cd project-1-manual-docker/frontend

# Build the image
docker build -t taskflow-frontend:1.0 .

# Run the container
docker run -d \
  --name taskflow-frontend \
  --network taskflow-network \
  -p 3000:3000 \
  taskflow-frontend:1.0
```

Step 7: Open the App

Open your browser: **`http://localhost:3000`**

Stopping Everything (Manual)

```
# Stop containers
docker stop taskflow-frontend taskflow-backend taskflow-db

# Remove containers
docker rm taskflow-frontend taskflow-backend taskflow-db

# Remove network
docker network rm taskflow-network

# (Optional) Remove volume to reset database
docker volume rm taskflow-pgdata
```

🐙 Project 2 — Docker Compose Setup

Step 1: Start Everything with ONE Command

```
cd project-2-docker-compose

# Build images and start all services
docker-compose up --build
```

That's it. ✅

Step 2: Run in Background

```
# Detached mode
docker-compose up --build -d

# Check status
docker-compose ps

# View combined logs
docker-compose logs -f

# View logs for specific service
docker-compose logs -f backend
```

Step 3: Open the App

Open your browser: <http://localhost:3000>

docker-compose.yml Explained

```
services:
  database:          # ← Service name (used as hostname)
    image: postgres:16-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data          # Named volume
      - ./database/init.sql:...initdb.d/:ro      # Auto-init script
    healthcheck:     # ← Compose waits for this before starting backend
      test: pg_isready

  backend:
    build: ./backend # ← Builds from Dockerfile
    depends_on:
      database:
        condition: service_healthy # ← Waits for DB to be READY ✅

  frontend:
    build: ./frontend
    depends_on:
      - backend          # ← Starts after backend
```

💡 **Tip:** The `depends_on` with `condition: service_healthy` ensures the backend only starts **after PostgreSQL is fully ready**, not just after the container starts.

Key Differences: Manual vs Compose

Feature	Manual Docker	Docker Compose
Network creation	<code>docker network create</code>	Automatic
Service discovery	Container names	Service names
Startup order	Manual	<code>depends_on</code>
Environment vars	<code>-e</code> flags per container	<code>environment:</code> block
Volumes	<code>-v</code> flag per container	<code>volumes:</code> block
Start all	3 separate <code>docker run</code>	<code>docker-compose up</code>
Stop all	3x <code>docker stop</code> + <code>rm</code>	<code>docker-compose down</code>

Stopping Compose

```
# Stop and remove containers (keep data)
docker-compose down

# Stop and remove containers AND volumes (reset database)
docker-compose down -v

# Stop and remove everything including images
docker-compose down -v --rmi all
```



Docker Command Cheat Sheet

Container Management

```
docker ps                # List running containers
docker ps -a             # List ALL containers
docker start <name>      # Start stopped container
docker stop <name>       # Gracefully stop container
docker restart <name>    # Restart container
docker rm <name>         # Remove stopped container
docker rm -f <name>      # Force remove (even running)
docker rename <old> <new> # Rename container
```

Image Management

```
docker images            # List images
docker build -t name:tag . # Build from Dockerfile
docker rmi <image>       # Remove image
docker image prune       # Remove dangling images
docker system prune -a   # ⚠️ Remove ALL unused resources
```

Network Management

```
docker network ls        # List networks
docker network create <name> # Create bridge network
docker network inspect <name> # Show network details
docker network connect <n> <c> # Connect container to network
docker network disconnect <n> <c> # Disconnect from network
docker network rm <name>    # Remove network
```

Volume Management

```
docker volume ls        # List volumes
docker volume create <name> # Create volume
docker volume inspect <name> # Show volume details
docker volume rm <name>    # Remove volume
docker volume prune      # Remove unused volumes
```

Docker Compose

```
docker-compose up          # Start services
docker-compose up -d       # Start in background
docker-compose up --build  # Build and start
docker-compose down        # Stop and remove
docker-compose down -v     # Stop, remove, and delete volumes
docker-compose ps          # List compose services
docker-compose logs -f     # Follow all logs
docker-compose exec backend sh # Shell into service
docker-compose restart backend # Restart one service
docker-compose build backend # Rebuild one service
```